RAFFLES INSTITUTION
H2 Computing (9569)
2025 Year 5

Tutorial S1B: Searching Algorithms (Solutions)

Section A: Discussion Questions

1.

Linear search: would take too long to search
- because there is no indication of where to start
- because there is no indication that data (name) is in order, hence must search entire list to confirm if student name is not in array.

Binary search: requires array to be sorted into alphabetical order.
- Continual halving and taking appropriate half.
- Mean length of linear search is 1000 (2000/2) searches.
- Maximum length of binary search is 11 (2^11 = 2048) searches.
- Non existence in array makes the length of linear search increase to 2000, while binary search remains at 11.

2.

(a) Using one-base indexing. searchKey = 60
List = [4, 9, 20, 24, 30, 34, 38, 42, 53, 56, 58, 60, 68, 71, 79, 82, 85, 90, 93, 95, 99]

low	high	low<=high	mid	arr[mid]	Return
1	21	True	11	58	
12		True	16	82	
	15	True	13	68	
	12	True	12	60	True

While loop executed 4 times.

(b) Using one-base indexing. searchKey = 35
List = [4, 9, 20, 24, 30, 34, 38, 42, 53, 56, 58, 60, 68, 71, 79, 82, 85, 90, 93, 95, 99]

low	High	low<=high	mid	arr[mid]	Return
1	21	True	11	58	
	10	True	5	30	
6		True	8	42	
	7	True	6	34	
7		True	7	38	
	6	False			False

While loop executed 5 times.

3.
(a)

	Alphabetical order of surname	Unsorted array
To search for a particular name	Will be faster with binary search, esp with 10000 names. O($\log_2 n$). Will be slower with linear search, since the name could be anywhere in the array from first item to last item. O(n).	Cannot use binary search since unsorted. With linear search, will be slower than searching on ordered array, because need to search entire array before confirming if name is stored in array.
To insert new names	Need to sort after insertion, hence will be slower than inserting without order.	Can append at the end, so much faster to insert.
To delete a name which has already been located	After deleting, may need to sort to maintain order, hence slower than deleting from unordered array.	Can delete from location and mark location as empty.
To print the whole list in alphabetical order of surname	Since already sorted, will be efficient to print.	Need to sort before printing, hence takes longer time than on a sorted array.

3(b) Frequency of search, insert, delete, print.
If need to search frequently, then better to store in alphabetical order and use binary search.
If need to insert frequently, then better to use unsorted array and maybe sort when needed for faster search with binary search.
If need to delete frequently, then better to store in sorted array and search for item to be deleted using binary search.
If need to print frequently, then better to store in alphabetical order.
Hence it depends on the frequency of each operation.

Section B: Assignment Questions

1.(a)

Identifier	Description
userList[1..20]	1D array to store userIDs
passwordList[1..20]	1D array to store passwords
maxIndex	Highest index of the userList and passwordList
myUserID	userID entered to login
myPassword	Password entered by user to login
userIDFound	False if userID not found in userList True if userID found in userList
loginOK	False if myPassword does not match password found in passwordList True if myPassword matches password found in passwordList
index	Pointer to current array element

(b) Complete the pseudocode for the Sam's algorithm:

```
maxIndex ← 20
INPUT myUserID
INPUT myPassword
userIDFound ← FALSE
loginOK ← FALSE
index ← 0
REPEAT
    index ← index + 1
    IF userList[index] = myUserID  THEN
        userIDFound ← TRUE
    ENDIF
UNTIL userIDFound = TRUE OR index = maxIndex
IF userIDFound = TRUE  THEN
    IF passwordList[index] = myPassword  THEN
        loginOK ← TRUE
    ENDIF
ENDIF

IF loginOK = TRUE  THEN
    OUTPUT "Login successful"
ELSE
    OUTPUT "UserID and/or password incorrect"
ENDIF
```

2.(a) First few stages of linear search for 888:

Start with 1. compare 1>888. False.
Compare next number 3. 3 > 888. False.
Compare next number 5. 5 > 888. False.
Compare next number 7. 7 > 888. False.
Compare next number 11. 11>888. False.
This carries on until compare with 889, 889 > 888. True.
Stop search and conclude that 888 is not found in the list.

First few stages of binary search:

Find middle number from 1 to 999, altogether 500 numbers, middle position is 250, which has number 499.
Compare 499 with 888. Not equal.
Is 499 < 888? Yes. Search top half from 501 to 999.

Find middle number from 501 to 999, altogether 250 numbers, middle position is 375, which has number 749.
Compare 749 with 888. Not equal.
Is 749 < 888? Yes. Search top half from 751 to 999.

Find middle number from 751 to 999, altogether 125 numbers, middle position is 438, which has number 875.
Compare 875 with 888. Not equal.
Is 875 < 888? Yes. Search top half from 877 to 999.

Find middle number from 877 to 999, altogether 62 numbers, middle position is 469, which has number 937.
Compare 937 with 888. Not equal.
Is 937 < 888? No. Search bottom half from 877 to 935.

Find middle number from 877 to 935, altogether 30 numbers, middle position is 453, which has number 905.

This search pattern continues until 888 is either found or the list of numbers runs out which means 888 is not found.

(b) Linear search through each element one at a time, from the first to the last. This means that if there are n elements, then it may take up to n comparisons to find the element.

Binary search halved the list each time it compares and search the appropriate half that may contain the number. This means that if there are n elements, then it may take up to log n comparisons to find the element.

Therefore binary search is much faster, since log n comparisons is much smaller than n when n is a very large number.

(c) Advantage of binary search vs linear search: much fewer searches as each comparison of element halved the list to be searched, compared with linear search which goes through every single element. Hence O(log n) for binary search vs O(n) for linear search.

Disadvantage: Need to sort the list before binary search can be applied.